



GitHub Trending Top 10

每日热门开源项目深度解读

2026-09-27

今日榜单

1	paperclipai/paperclip	TypeScript	3天
2	vectorize-io/hindsight	Python	4天
3	debpalash/VoiceStudio	Python	NEW
4	rohitg00/ai-engineering-from-scratch	Python	2天
5	InfinityLoop1308/PipePipe	Shell	NEW
6	vercel-labs/scriptc	TypeScript	NEW
7	mvschwarz/openrig	TypeScript	NEW
8	dream-num/univer	TypeScript	6天
9	willfaust/Madeira	C	NEW

#1

TypeScript

连续 3 天

paperclip

The open-source app everyone uses to manage agents at work

Stars **88,807** Forks **15,568** Today **+2,527**

<https://github.com/paperclipai/paperclip>

项目简介

Paperclip 是一个开源的 AI Agent 编排平台，定位是"AI 员工的公司管理层"——如果 OpenClaw 等单个 Agent 是员工，Paperclip 就是让这些员工协同工作的公司。它解决的核心问题是：当团队同时运行多个 AI Agent（Claude Code、Codex、Cursor 等）时，如何统一分配目标、协调工作、监控成本和审计产出。

核心功能

- **目标驱动的任务管理**：用户定义业务目标（如"打造第一的 AI 笔记应用做到 100 万美元 MRR"），Agent 团队自主拆解执行，用户只负责审批策略和验证结果，而非管理每个 PR。
- **多 Agent 编排与组织架构**：支持接入 OpenClaw、Claude Code、Codex、Cursor、Bash、HTTP 等多种 Agent 类型，可像组建公司一样为它们分配 CEO、CTO、工程师、设计师等角色。
- **预算与成本治理**：内置预算控制和成本追踪，可在仪表盘中监控各 Agent 的开销，防止自主运行失控烧钱。
- **审批与审计机制**：提供审批关卡、审查门（review gates）和可审计的工作流，让 Agent 7×24 自主运行的同时保留人工介入和事后追溯的能力。

适用场景

主要面向两类用户：一是同时开着几十个 AI 终端、已经"管不过来"的重度 Agent 用户；二是想构建自主 AI 组织（autonomous AI organizations）的创业者或团队，希望让 Agent 团队持续运转一个业务。典型场景包括自主运营产品、批量内容生产、多 Agent 协作开发等，README 特别强调"从手机管理你的自主业务"。

技术亮点

技术栈是 Node.js 服务端 + React 前端 UI，采用 TypeScript 编写，以 MIT 协议开源。设计上最有意思的是它的抽象层次——把 Agent 编排建模为"公司治理"：org chart（组织架构）、budgets（预算）、

governance（治理）、goal alignment（目标对齐），这套元模型比单纯的 task queue 更贴近真实团队协作。此外"只要能接收 heartbeat 就能被雇佣"的设计，意味着它对 Agent 接入方式保持了高度开放性。

上手建议

建议从 README 中的 Quickstart 和官方文档（docs.papercliping）入手，先在本地跑通一个最小 Agent 团队，体验"定义目标 → 组建团队 → 审批运行"的完整流程。想深入贡献的话，可以加入其 Discord 社区了解开发动态；由于项目星标增长极快（单日 +2527），生态和文档可能仍在快速迭代，建议关注 GitHub issues 和 release 节奏，从文档完善或 Agent 适配器这类边界清晰的模块切入贡献。

#2

Python

连续 4 天

hindsight

Hindsight: Agent Memory That Learns

Stars **35,783** Forks **4,536** Today **+4,463**

<https://github.com/vectorize-io/hindsight>

项目简介：Hindsight 是 Vectorize 推出的 Agent 记忆系统，目标是让 AI Agent 不只是"记住"对话历史，而是能像人一样从经验中"学习"。它针对 RAG 和知识图谱在长期记忆任务上的不足，主打"会学习的记忆"，并在 LongMemEval 基准上宣称达到 SOTA 水平。

核心功能： - **三种核心操作：** retain（保留）、recall（召回）、reflect（反思），覆盖记忆的写入、检索和提炼全流程。 - **多种记忆形态：** 支持 observations（观察）、mental models（心智模型）、knowledge pages（知识页）和 memory banks（记忆库），让记忆结构化、可沉淀。 - **极简接入：** 通过 LLM Wrapper 两行代码即可集成，同时提供 Python/JS 客户端、MCP Server 及多种 coding agent 集成。 - **灵活部署：** 既有 Docker 一键启动的服务端，也支持 Python 嵌入式（无需服务器），并兼容 25+ LLM provider（含本地 Ollama）。

适用场景： 适合构建需要长期记忆能力的对话式 Agent、个人助理、客服机器人、编码助手等。企业级用户（README 提到 Fortune 500 已在生产使用）可用于需要跨会话积累知识的场景；开发者也可用嵌入式模式快速做原型验证。

技术亮点： - 用"学习"而非单纯"检索"的范式重新定义 Agent 记忆，区别于传统 RAG 和知识图谱路线。 - 在 LongMemEval 上的成绩由 Virginia Tech Sanghani Center 和华盛顿邮报独立复现，可信度较高（其他厂商多为自报数据）。 - 提供实时更新的 benchmark 站点，公开 per-model 的准确率、延迟和成本，工程透明度好。

上手建议： 最快路径是跑一遍 Docker 命令（设置 OPENAI_API_KEY 后一键启动，UI 在 9999 端口），或直接用 Python 嵌入式模式免服务端试用。建议先阅读官方 Cookbook 和 Core Concepts 理解 retain/recall/reflect 的用法，再参考 Integrations 接入自己的 Agent；贡献方面可关注 GitHub Issues 和 Slack 社区，文档 skill 也已支持在 Claude Code、Cursor 里边写边查。

#3

Python

NEW

VoiceStudio

VoiceStudio is the open-source, fully-local ElevenLabs alternative — voice cloning, voice design, video dubbing, dictation, transcription & audiobook creation in 646 languages.

Stars **38,990** Forks **4,639** Today **+3,060**

<https://github.com/debpalash/VoiceStudio>

项目简介：VoiceStudio 是一个完全本地运行的开源语音 AI 工作台，定位为 ElevenLabs 的开源替代品。它把语音克隆、语音设计、视频配音、听写、转录和有声书制作整合到一个桌面应用里，解决的是商业语音服务价格高、数据需上传云端、隐私不可控的问题。项目用 Python 为主开发，配合 Electron 桌面端，目前在 GitHub 上已获得近 3.9 万星标，单日新增超 3000，热度极高。

核心功能：一是语音克隆与语音设计，用户可以上传一段干净参考录音复刻音色，或用文字描述凭空"设计"一个声音；二是视频配音，支持带时间轴的对齐语音合成；三是听写与转录，配有悬浮小部件方便随时记录；四是有声书和批量任务生成；五是本地 API 与 MCP 接口，可让 AI Agent 直接调用语音能力。

适用场景：内容创作者可以用它给视频配音、制作有声书；开发者和 AI Agent 构建者可以通过本地 API/MCP 把语音能力接入自己的应用；注重隐私的用户（如处理敏感音频的个人或团队）可以在自己硬件上完成全部处理，无需上传云端。646 种语言的支持也让它适合多语言内容生产。

技术亮点：默认引擎基于 k2-fsa/OmniVoice，且架构上支持切换多种语音引擎，避免绑定单一模型；完全本地推理，远程服务为可选而非必需，分析数据需用户明确同意才会上报；提供 MCP 协议支持，顺应了 AI Agent 生态的工具调用趋势；安装方式也很有特色，支持一行命令安装，甚至可以直接让 Claude Code、Cursor 等编码 Agent 按文档自动完成安装和验证。

上手建议：最快的方式是 macOS/Linux 下执行 `curl -fsSL https://voicestudio.sh/install | sh` 一键安装，或从 Releases 页面下载对应平台版本。想先体验功能，打开"Voice cloning"工作区选一个音色或添加参考录音即可生成。想从源码运行或参与贡献，README 中提供了 Electron 预览的源码运行说明，建议先阅读 docs/feature-catalog.md 了解引擎目录，并注意项目采用 AGPL-3.0 许可证，商用需留意其开源义务。

#4

Python

连续 2 天

ai-engineering-from-scratch

Learn it. Build it. Ship it for others.

Stars **58,884** Forks **10,193** Today **+848**

<https://github.com/rohitg00/ai-engineering-from-scratch>

项目简介

ai-engineering-from-scratch 是一份面向 AI 工程化的开源课程体系，目标是弥合"学生会用 AI 工具"与"能专业地构建 AI 系统"之间的巨大鸿沟（README 引用数据：84% 学生用过 AI 工具，仅 18% 自认能专业使用）。它不是零散教程，而是一套从数学基础到 Agent 工程、覆盖 20 个阶段共 523 节课的完整学习路径。作者是知名项目 Agent Memory 的创作者 Rohit G00，项目采用 MIT 协议免费开放。

核心功能

- **分阶段课程体系**：20 个 phase 从环境搭建、数学基础、ML，一路延伸到 LLM 工程、工具协议（MCP）、Agent 工程，结构清晰、循序渐进。
- **多语言实现**：课程代码同时提供 Python、TypeScript、Rust、Julia 四种语言版本，覆盖不同技术栈的开发者。
- **可复用产物导向**：每节课都产出一个可复用 artifact——prompt、skill、agent 或 MCP server，强调"学完即用"。
- **目标驱动的入口设计**：提供"我想构建 X"的导航表，学习者可按目标（如构建生产级 LLM 应用、构建 Agent）直接跳转，无需通读全部课程。
- **多语言本地化**：README 提供 12 种语言版本，课程页面通过 translations 分支机器翻译，降低非英语用户门槛。

适用场景

- **AI 工程新人**：想系统补齐从数学到工程落地的完整知识链，而非零散看博客。
- **有 Python 基础但缺 ML/LLM 背景的开发**：可直接从 Phase 1 数学基础或 Phase 11 LLM 工程切入。
- **想上手 Agent / MCP 的工程师**：Phase 13、14 提供了从 Agent Loop 到 MCP 协议的实战路径。

- **团队培训或自学路线参考**：20 阶段的结构可直接作为学习计划模板。

技术亮点

- **"By hand"理念**：强调端到端手写实现，而非调用现成 API，帮助理解底层机制。
- **多语言并行**：同一课程用四种语言实现，既照顾不同技术背景，也便于跨语言对比学习。
- **文档与代码同源**：GitHub 与官网共享同一套 lesson code，避免版本漂移。
- **数据驱动维护**：README 中的阅读量统计由 `site/stats.json` 经 `build.js` 自动生成，工程化程度较高。

上手建议

- **新手**：从 Phase 0（Setup and Tooling）开始，先搭好开发环境，再按 phase 顺序推进。
- **有目标的学习者**：直接看 README 顶部的"choose what you want to build"表格，按目标跳转到对应 phase 或 learning path。
- **贡献者**：先读 `docs/i18n.md` 了解翻译流程，课程内容以英文为准（canonical），翻译在 `translations` 分支；也可参考 `ROADMAP.md` 了解待补内容。
- **验证效果**：官网 `aiengineeringfromscratch.com` 与 GitHub 内容一致，可任选其一学习。

#5

Shell

NEW

PipePipe

An open-source Android app to let you browse YouTube and other services freely.

Stars **6,431** Forks **214** Today **+139**

<https://github.com/InfinityLoop1308/PipePipe>

项目简介：PipePipe 是一个基于 NewPipe 深度改造的开源 Android 应用，让用户无需 Google 服务即可自由浏览 YouTube、Bilibili 等视频平台。它解决了官方客户端广告多、无法后台播放、缺乏定制化等痛点，同时通过硬分叉策略实现了比原版 NewPipe 更快的迭代和更丰富的功能。

核心功能：集成 SponsorBlock 自动跳过赞助片段，并通过 ReturnYouTubeDislike 恢复被隐藏的踩数；支持登录以访问受限或会员内容，同时提供 AV1/VP9 高效解码和弹幕式直播聊天覆盖；内置关键词/频道过滤、Shorts 和付费视频屏蔽、滑动跳转、长按倍速、睡眠定时器等播放控制；增强的播放列表管理支持整列表下载和本地搜索排序。

适用场景：适合不想被广告和推荐算法干扰、希望在移动端后台听音乐或看视频、需要下载内容离线观看的 Android 用户。尤其适合经常访问 YouTube 和 Bilibili、对隐私和定制化有较高要求的用户，以及希望摆脱 Google 服务框架的极简主义者。

技术亮点：项目采用硬分叉模式，不跟随 NewPipe 上游更新，从而能快速修复问题并频繁推出新功能；登录 Cookie 仅在指定场景（如获取播放流）中使用，兼顾了功能与隐私；使用 Shell 作为主要语言暗示其构建和自动化脚本较为轻量，同时支持 AV1/VP9 等现代编解码器，体现了对播放性能的重视。

上手建议：普通用户可直接从 F-Droid 或 IzzyOnDroid 安装 APK 体验。开发者若想贡献代码，建议先阅读 PipePipe Wiki 了解项目结构，然后从修复小问题或实现社区需求入手提交 PR；注意作者明确不接受服务请求，有意新增平台支持者应自行 fork 仓库独立开发。

#6

TypeScript

NEW

scriptc

TypeScript-to-Native Compiler

Stars **5,261** Forks **136** Today **+76**<https://github.com/vercel-labs/scriptc>

项目简介：scriptc 是 Vercel Labs 推出的实验性编译器，能把 TypeScript/JavaScript 直接编译成原生可执行文件、C 代码、LLVM IR、汇编或 WebAssembly。它解决的是"用 TS 写代码、却要跑在无 Node 环境"的需求，让 JS 生态的代码能脱离运行时独立部署。

核心功能： - 多级输出：一条命令可产出 typed IR、可读 C、LLVM IR、汇编、目标文件和原生可执行文件，产物累积在 .scriptc/ 目录。 - 静态编译：内置小型原生运行时，生成的二进制不依赖 Node 或 JS 引擎；无法静态编译的代码会以诊断形式报告。 - 动态回退：对 npm 包和 any 类型代码，用 --dynamic 显式嵌入 quickjs-ng 引擎兜底。 - 覆盖率检查：scriptc coverage 量化程序有多少能静态编译，并给出每条动态/不支持代码的诊断编号。 - Node API 支持：node:http 等部分 Node API 可直接编译到原生运行时。

适用场景：适合想把 TS 工具链、CLI、服务端程序编译成零依赖原生二进制的开发者；也适合探索 JS 到原生/WASM 编译路径的研究者，或在 WASI 环境下部署 TS 代码的场景。由于是实验项目，不建议用于生产关键系统。

技术亮点：复用 TypeScript 官方编译器做解析和类型检查，避免自造前端；在 macOS 15+ arm64 上普通 LLVM 层可执行文件只需 clang 作为链接驱动，实际编译用自带 helper 和预编译运行时包，无需 C 编译器；外部目标文件消费通过版本化的 --print=native-link-info JSON 配方描述 ABI 和依赖，设计上刻意不暴露隐藏缓存路径。

上手建议：先 npm install -g scriptc (需 Node 24+)，用 README 里的 hello.ts 跑 scriptc run 和 scriptc build 感受流程，再用 --emit=c/--emit=llvm 观察各层产物。想贡献可从 scriptc coverage 报告的动态代码入手，扩展静态覆盖或补充 Node API 到原生运行时的映射。

#7

TypeScript

NEW

openrig

Multi-agent harness that runs Claude Code and Codex together as one system

Stars **690** Forks **82** Today **+114**

<https://github.com/mvschwarz/openrig>

项目简介：OpenRig 是一个多智能体编排框架（multi-agent harness），它把 Claude Code 和 Codex 这两个独立的 AI 编码工具整合成一支统一管理的"团队"。它解决的问题是：以往 AI 编码助手只是散落的一堆终端会话，缺乏持久化组织、协作和上下文延续，OpenRig 让它们像真正的开发团队一样持续工作。

核心功能：

- **YAML 定义团队：**用声明式配置定义智能体团队（如 owner 和 checker 角色），一条命令即可启动整个 rig。
- **Lead Agent 协调机制：**与一个"主控"智能体对话描述目标，由它跨团队协调专家、汇总结果并上报需要你决策的事项。
- **跨 Harness 混编：**在同一系统中同时运行 Claude Code 和 Codex，统一调度而非各自为战。
- **TUI 共享仪表盘：**通过 `rig tui --shared` 查看团队运行状态（图/表格/席位详情），支持分离后重新接入。
- **任务队列与消息：**`rig send`、`rig queue list` 等命令实现任务派发、追踪和结果回收。

适用场景：适合需要让 AI 智能体长时间、有组织地处理代码库任务的开发者，尤其是希望"一个仓库持续迭代、保持上下文不丢失"的场景。典型用法是给 owner 智能体下达一个有边界的改动目标，由 checker 审查后再回到同一 owner 做下一步，形成可循环的协作流程。

技术亮点：项目用 TypeScript 编写，基于 tmux 管理终端会话，通过 SQLite 持久化状态；它主动写入 provider hooks 和 workspace trust 设置来打通与 Claude Code/Codex 的集成，这种"包裹 harness 的 rig"抽象层设计是核心创新。README 特意强调"OpenRig 会修改你机器上的哪些内容"，体现了对安全边界的透明态度。

上手建议：先确保 Node.js 20/22/24 和 tmux 就绪，运行 `npm install -g @openrig/cli` 后用 `rig setup --dry-run` 预览改动再正式应用。建议从官方 guided first-use 文档（docs/reference/getting-started.md）走一遍"就绪检查→下达任务→审查结果"的完整路径；贡献者可先阅读 CONTRIBUTING.md，项目承诺一天内响应 issue 和 PR。

#8

TypeScript

连续 6 天

univer

The Office Harness for AI Agents — Spreadsheets, Docs, Slides, Canvas, Relational Tables, and PDF in one runtime.

Stars **19,949** Forks **1,694** Today **+920**

<https://github.com/dream-num/univer>

项目简介：Univer 是一个开源的 Office SDK，让开发者能把电子表格、文档、演示文稿等办公能力直接嵌入自己的产品中。它解决的核心问题是：过去要自建类 Excel/Google Docs 的编辑体验，要么依赖笨重的托管服务，要么从零造轮子，而 Univer 提供了一套可插拔、可定制的底层框架。

核心功能：一是覆盖表格、文档、幻灯片、看板、关系表等多形态办公内容，并计划支持 PDF；二是提供统一的 Facade API，同一套代码可在浏览器和 Node.js 服务端运行；三是内置公式引擎与 Canvas 渲染，支持高性能的大数据量渲染；四是采用插件化架构，可按需组合功能，也可用预设快速起步；五是支持自定义插件、命令、服务和 UI 组件来扩展行为。

适用场景：适合需要在 SaaS 产品、内部工具、BI 工作流或 AI 应用中嵌入表格/文档编辑能力的团队。典型用法包括为 AI Agent 提供可读写的工作文件环境、构建自托管的协作办公空间（如官方示例 Univer Workspace）、以及在服务端做 workbook 批处理。

技术亮点：最值得关注的是“同一运行时贯穿浏览器与 Node.js”的设计，让前后端共享存储与计算逻辑；Canvas 渲染 + 公式引擎的组合，使其能处理复杂表格而不牺牲性能；插件架构和 Facade API 把“框架”与“成品应用”解耦，开发者既能快速用预设，也能深度定制。此外，它明确把“人与 AI Agent 在同一文件中协作”作为产品方向，这在办公 SDK 里是较新的定位。

上手建议：建议先访问官网 univer.ai 和文档 docs.univer.ai，跑通一个最小嵌入示例，再对照 Showcase 和 Univer Workspace 参考实现理解集成方式。想贡献代码的话，可从 GitHub 的 issues 和 dev 分支入手，先熟悉插件与命令体系，再针对具体模块提交 PR。

#9

C

NEW

Madeira

Run x86-64 Windows PC games on jailed iOS via FEX-Emu + Wine + DXMT

Stars **722** Forks **153** Today **+171**

<https://github.com/willfaust/Madeira>

项目简介：Madeira 是一个让非越狱 iPhone 运行 x86-64 Windows PC 游戏的研究性项目。它通过把 Wine、FEX-Emu 和 DXMT 三层技术整合进单一 Mach 进程，在 iOS 上模拟完整的 Windows 游戏运行环境，目标是让玩家在手机上直接跑 PC 游戏。

核心功能： - x86-64 → ARM64 指令翻译（基于 FEX-Emu），解决 iOS 设备 ARM 架构无法直接执行 x86 代码的问题。 - 通过 Wine（ARM64EC 构建）提供 Windows API 兼容层，让 Windows 游戏无需修改即可运行。 - 用 DXMT 把 D3D11 调用转译为 Metal，直接对接 iOS 的图形栈。 - wineserver 以线程而非独立进程方式运行，规避 iOS 对多进程的严格限制。 - 利用 JIT + 调试器附加（StikDebug）绕过 iOS 的代码执行限制。

适用场景：适合喜欢折腾的 iOS 玩家、模拟器爱好者和移动端逆向研究者，用于在 iPhone 上体验部分 PC 游戏（目前 Thumper、ULTRAKILL 可玩）。由于需要侧载、每周重签名以及调试器附加，普通用户门槛较高，更适合技术尝鲜或研究用途。

技术亮点：最值得关注的是“单 Mach 进程 + wineserver 线程化”的设计，这是绕开 iOS 沙盒与多进程限制的关键。其次是三套异构技术（Wine/FEX/DXMT）的深度整合，以及对上游项目的 iOS 化 fork 和重新许可处理（如 Wine 依 LGPL § 3 relicensed 为 GPL-3.0）。README 还明确标注了 AI 辅助代码不得回馈上游的合规边界，这在开源项目中较为少见。

上手建议：先阅读 README 的 Requirements 和 Building 部分，准备好非越狱 iPhone、StikDebug 和 Apple ID（免费账号即可，但需每周重签）。克隆时必须带 `--recurse-submodules`，因为 FEX、wine、dxmt 都指向作者自己的 fork，上游版本无法构建。想贡献代码的话，注意各 fork 的许可证差异（见 LICENSE-MADEIRA.md），并遵守上游的 AI 贡献政策，避免把 AI 生成代码提交回 FEX-Emu 等上游项目。

数据来源: github.com/trending

报告日期: 2026-09-27

由 DeepSeek AI 提供智能分析 | 自动生成